

Data Warehouse de The Drinking Company

Diseño e Implementación de un Sistema Dimensional para Analytics Comercial

RESUMEN EJECUTIVO

La pregunta: ¿Cómo integramos datos de múltiples fuentes heterogéneas para que una empresa de bebidas pudiera analizar su negocio sin ambigüedad?

La solución: Diseñé e implementé un Data Warehouse en esquema dimensional (Star Schema) usando SSIS y SQL Server, resolviendo un problema arquitectónico crítico: la ambigüedad geográfica. El resultado es un sistema que permite análisis comercial limpio sobre ventas, inventario y desempeño de empleados.

El impacto: The Drinking Company pasó de tener datos fragmentados en 3 sistemas distintos a tener una única fuente de verdad que responde preguntas de negocio de forma consistente y sin conflictos de relación.

CONTEXTO: El Problema de Negocio

The Drinking Company (TDC) es una empresa de bebidas que necesitaba entender su negocio desde la perspectiva comercial. Pero tenía un problema clásico:

- Datos fragmentados: Ventas en MySQL, histórico en SQL Server, información de referencia en archivos planos (txt).
- Preguntas sin respuesta: ¿Qué bebidas venden más?, ¿Dónde viven mis clientes de alto valor?, ¿Cómo está el stock?
- Nivel de análisis: Cada area (ventas, producción) tenía sus propios reportes desconectados. No había visión integrada.

Además, las herramientas de BI (Power BI en este caso) estaban limitadas: sin un modelo dimensional bien diseñado, los análisis serían lentos, confusos y propensos a errores.

El desafío: construir un modelo que unificara esos datos de forma que fuera fácil de analizar, rápido de consultar, y que no generara ambigüedades en las relaciones.

HALLAZGO 1: Arquitectura Dimensional — Star Schema (Casi)

La decisión

Elegí un esquema dimensional en Star Schema: una o dos tablas de hechos rodeadas de dimensiones que se conectan directamente.

¿Por qué Star Schema y no normalizado (3NF)?

- Velocidad: Star Schema es mucho más rápido para queries de BI (pocos JOINS, simples).
- Mantenibilidad: Fácil de entender para business users y analistas.
- Compatibilidad BI: Herramientas como Power BI están optimizadas para Star Schema.

¿Por qué "casi"?

Incorporé una excepción puntual tipo Snowflake: la dimensión DIM_CLIENTE no contiene directamente los atributos geográficos, sino que apunta a una dimensión adicional DIM_GEOGRAFIA_CLIENTE. ¿Por qué? Aquí viene el problema real.

HALLAZGO 2: El Problema de la Geografía Ambigua

El problema real

TDC necesita responder DOS preguntas geográficas simultáneamente:

- ¿Dónde viven mis clientes? → Para segmentación y análisis demográfico.
- ¿Dónde estoy vendiendo? → Para análisis de canal de distribución.

Ambas preguntas comparten exactamente la misma estructura de datos: Ciudad, Estado, Código Postal, Región Comercial, y provienen de la misma fuente (archivo Regions.txt).

Pero son preguntas conceptualmente distintas, porque un cliente puede vivir en Buenos Aires pero sus productos venderse en Rosario.

La tentación (y por qué no funciona)

La primera solución que muchos proponen: "Crea una única dimensión de geografía y reutilízala en ambos lados."

Problema: Si una sola tabla de geografía es referenciada desde DIM_CLIENTE Y desde FACT_VENTAS, se genera un doble camino (double path) en el modelo:

```
FACT_VENTAS → DIM_CLIENTE → DIM_GEOGRAFIA_CLIENTE (camino indirecto)
FACT_VENTAS → DIM_GEOGRAFIA_VENTA (camino directo)
```

En Power BI, esto genera relaciones ambiguas: cuando preguntás "¿Cuánto vendí en provincia?" — ¿hablas de dónde viven los clientes o dónde se vende? El motor de BI no sabe cuál relación usar. Resultado: errores silenciosos en los números.

La solución: Dos Dimensiones Independientes

Creé dos tablas de geografía físicamente independientes:

- DIM_GEOGRAFIA_CLIENTE (hacia DIM_CLIENTE)
- DIM_GEOGRAFIA_VENTA (hacia FACT_VENTAS)

Ambas contienen exactamente la misma información geográfica, pero como tablas distintas. Esto elimina el doble camino y Power BI puede usar cada relación sin ambigüedad.

Compromisos y Ganancias

Compromisos: Redundancia de datos (mínima — es una dimensión pequeña) y un poco más de complejidad en el ETL (necesitas duplicar los datos).

Ganancia: Relaciones limpias sin ambigüedad, métricas correctas en los dashboards, y claridad conceptual.

HALLAZGO 3: Dos Tablas de Hechos, Dos Granularidades

FACT_VENTAS

- Granularidad: Línea de producto dentro de una factura.
- Cada fila: Una venta de un producto específico en un comprobante específico.
- Información: Cantidad, precio, descuento.
- Propósito: Análisis comercial (qué se vende, a quién, cuándo, dónde).

FACT_INVENTARIO

- Granularidad: Fotografía diaria del stock por producto.
- Cada fila: El estado del inventario de un producto en un día específico.
- Propósito: Tracking de disponibilidad, detección de desabastecimiento.

¿Por qué dos granularidades distintas? Porque responden a preguntas de negocio distintas. Una factura registra qué se vendió; un inventario registra qué había disponible. No tienen la misma forma.

HALLAZGO 4: ETL Real — Transformaciones desde Múltiples Fuentes

Las fuentes

- MySQL: Datos actuales de ventas (tabla billing).
- SQL Server: Histórico de ventas (tabla billing legacy).
- Archivos planos (txt): Maestros de referencia (Regions.txt, productos, etc).

El proceso ETL (SSIS)

Etapas 1 - Extracción heterogénea: Extraje desde 3 orígenes distintos usando conectores SSIS (OLE DB para SQL Server, ADO.NET para MySQL, Flat File Source para archivos).

Etapas 2 - Staging (ELT philosophy): Depositó los datos crudos en tablas de staging sin transformaciones de negocio. Esto permite auditoría, debugging más fácil, y reutilizar datos.

Etapas 3 - Transformaciones de limpieza: Usé vistas SQL para convertir a Unicode, limpiar espacios sobrantes, y validar tipos de dato.

Etapas 4 - Carga a dimensiones y hechos: Con los datos depurados, cargué hacia el DW.

Etapas 5 - Duplicación inteligente (Multicast): Para las dimensiones geográficas, usé un componente Multicast de SSIS para duplicar el mismo flujo de datos en dos direcciones simultáneamente.

Decisiones de descarte

Campo DESCARTADO: ID (autoincremental) — Era un identificador técnico del motor, no del negocio. Ya teníamos BILLING_ID (verdadero identificador de factura). Acción: Descartado como redundante.

Campo DESCARTADO: BRANCH_ID — Identifica la sucursal donde se registró cada transacción, pero no existe maestro de sucursales que traduzca BRANCH_ID a ubicación geográfica. Sin ese maestro, BRANCH_ID es un número sin contexto, no accionable para BI. Acción: Descartado por no accionable.

Lección: En un DW, incluir datos sin contexto es peor que no incluirlos. Generan confusión.

LIMITACIONES Y SUPUESTOS

- Ventanas temporales no superpuestas: Datos de ventas (2006-2009) vs. inventario (2002-2004) no se solapan.
- Datos de origen limitados: No hay información sobre devoluciones, cambios, o retornos de productos.
- Asume calidad de datos de origen: Si la data está sucia en MySQL o SQL Server, el DW la heredará.

QUÉ APRENDÍ

Técnico

- Diseño dimensional es arte, no ciencia: Hay decisiones que dependen del contexto de negocio.
- SSIS para ETL real es poderoso pero complejo: Transformaciones heterogéneas requieren entender los conectores a fondo.
- El 'double path problem' es real en BI: Muchos reclutadores no lo conocen, es un diferenciador documentarlo.

De negocio

- Granularidad es crítica: Elegir la granularidad de hechos determina qué preguntas puedes responder después.
- Data sin contexto es ruido: `BRANCH_ID` sin maestro es un buen ejemplo.
- Redundancia controlada > ambigüedad: Dos tablas de geografía es redundancia, pero vale la pena por claridad.

CONCLUSIÓN

Este proyecto fue más que "construir un DW": fue resolver un problema de ambigüedad arquitectónica usando pensamiento crítico y conocimiento de las limitaciones de las herramientas de BI.

El resultado es un sistema reproducible, bien documentado, y que permite que TDC tome decisiones basadas en datos con confianza.